DROPOUT-AWARE TRAINING FOR SENSOR-ROBUST FAULT LOCALIZATION WITH SPATIO-TEMPORAL GRAPH NEURAL NETWORKS

P K SANGAMESWAR
Guide: M Anbazhagan
Amrita Vishwa Vidyapeetham

Problem Statement

- Distribution feeders are **sparsely instrumented**: μPMUs sit on a few buses only.
- Fault must be localized to a **bus**, not just detected — 25 candidate fault buses on IEEE-123.
- Only **6 of the 25 fault buses carry a μPMU → 19 fault locations are never directly measured**; the model must infer them from neighbours through the grid topology
- Real deployments make this worse: μPMUs **drop out** (comms loss, GPS-sync loss, power loss, maintenance). Published STGNN results all assume **full sensor availability**.

Research Gap

No published evaluation of what happens to STGNN fault localization when sensors go missing at inference time, and no mitigation for it.

Objectives

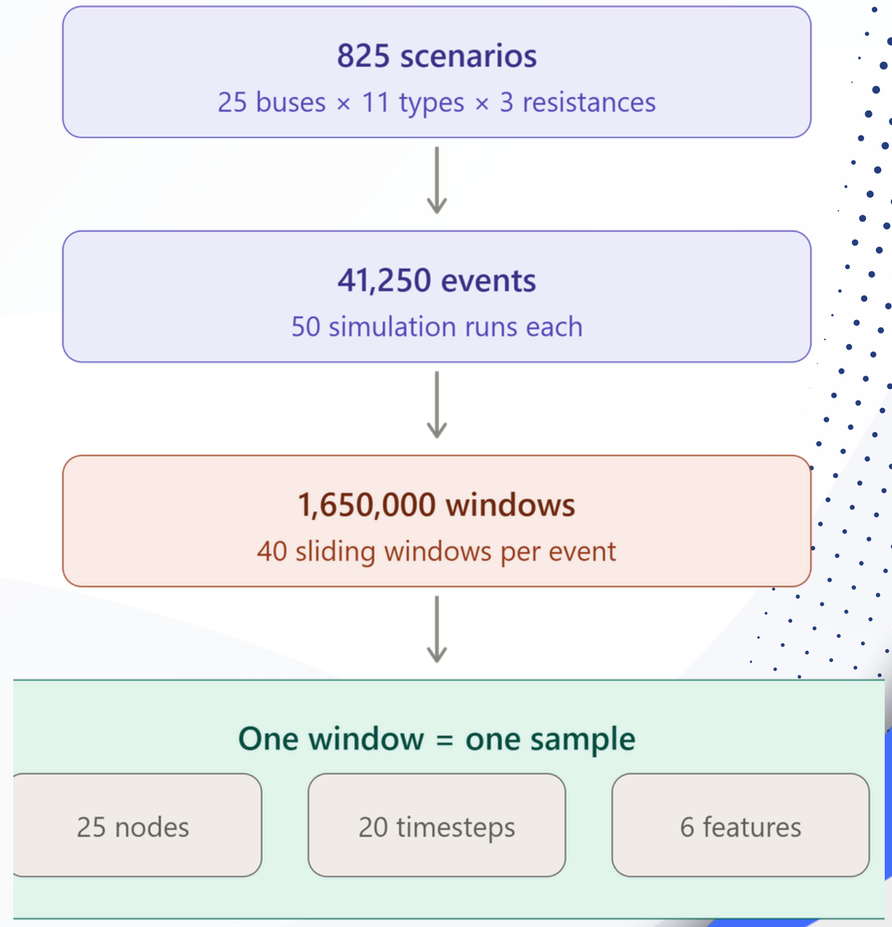
- **Adopt** the STGNN fault-localization methodology of Karabulut et al. (2026) on IEEE-123 under partial observability, and report **our own** numbers across 5 architectures × 3 seeds.
- **Quantify** robustness: measure accuracy degradation as $k = 0$, 1, 2, 5, 10 μPMUs are dropped.
- **Mitigate** it: propose and validate **dropout-aware training** (architecture-agnostic).
- **Ablate** an architectural alternative: **SDR-STGNN**, an availability-gated aggregator.
- **Stress-test** the mitigation against imputation, worst-case (targeted) loss, and realistic per-event (persistent) loss.

Related Work

Work	Approach / Architecture	Loss mitigation	Gap
Chen et al. (2020)	Spectral GCN (Chebyshev), static snapshot	✓ noise/data-loss augmentation	Single snapshot — no temporal axis
Zhang et al. (2025)	GDIA-GCL: Dirichlet imputation + contrastive GCN	✓ imputation	Unmeasured set assumed fixed & known in advance
Karabulut et al. (2026)	STGNN: GRU + RGCN / RGSAGE / RGATv2	—	Assumes full sensor availability at inference
This work	STGNN + node-dropout study	**✓ dropout-aware training**	**— (fills the gap)**

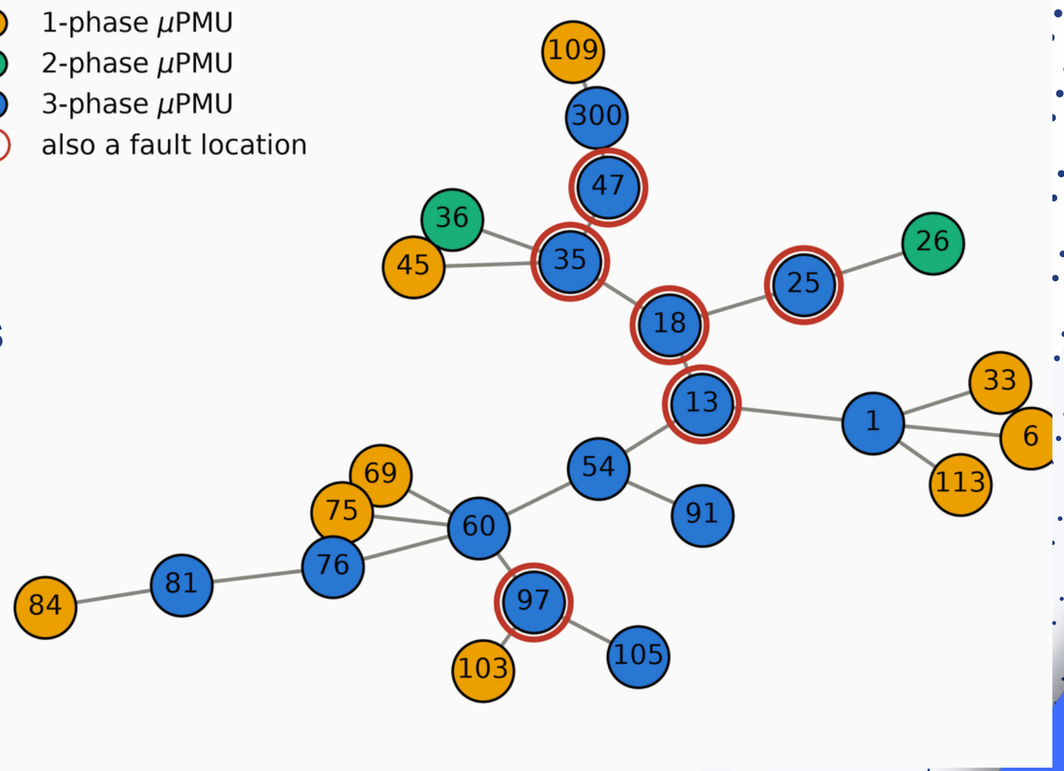
Dataset and Simulation Setup

- **Feeder:** IEEE 123-bus, 4.16 kV, 60 Hz — **OpenDSS** (OpenDSSDirect.py), Daily mode, **1 ms** step.
- **Factorial design:** 25 fault buses × 11 fault types (3 LG, 3 LL, 3 LLG, 2 three-phase) × 3 fault resistances (0.1, 1, 10 Ω) = **825 scenarios**.
- 50 runs/scenario, load multiplier ~ $\mathcal{U}(0.5, 1.3)$ per run → 41,250 events.
- Windowing: 60 ms record (40 ms pre-fault, 20 ms fault), 20-step windows → 40 windows/event → 1,650,000 windows total. Exactly 50 % no-fault by construction
- Split: event-level 70/15/15 → train 1,155,000 / val 248,000 / test 247,000 windows (6,175 test events). Verified 0 leakage — no window of an event appears in two splits.



Graph Construction

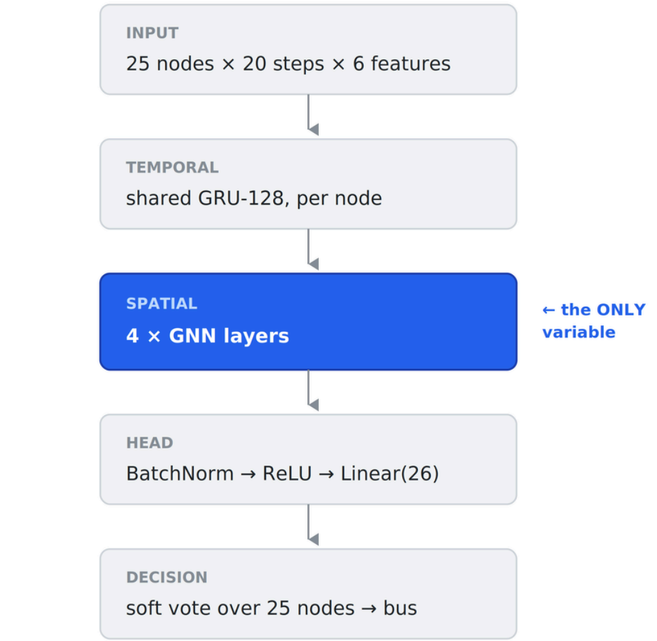
- Nodes = **25 µPMU buses only** (not all 123 buses).
- Edges built by a **substation-rooted spanning tree** over the measured set (Algorithm 1): two µPMUs are connected if the electrical path between them contains no other µPMU.
- Result: a **connected 25-node, 24-edge tree** — the graph is a topological abstraction of the feeder, so an unmeasured fault bus shows up as a *signature pattern across its measured neighbours*.
- **Mask channels are static, not all-ones:** only **14 of 25** µPMUs are three-phase; the rest show patterns like (1,0,0). They encode per-node phase instrumentation and never change over time.



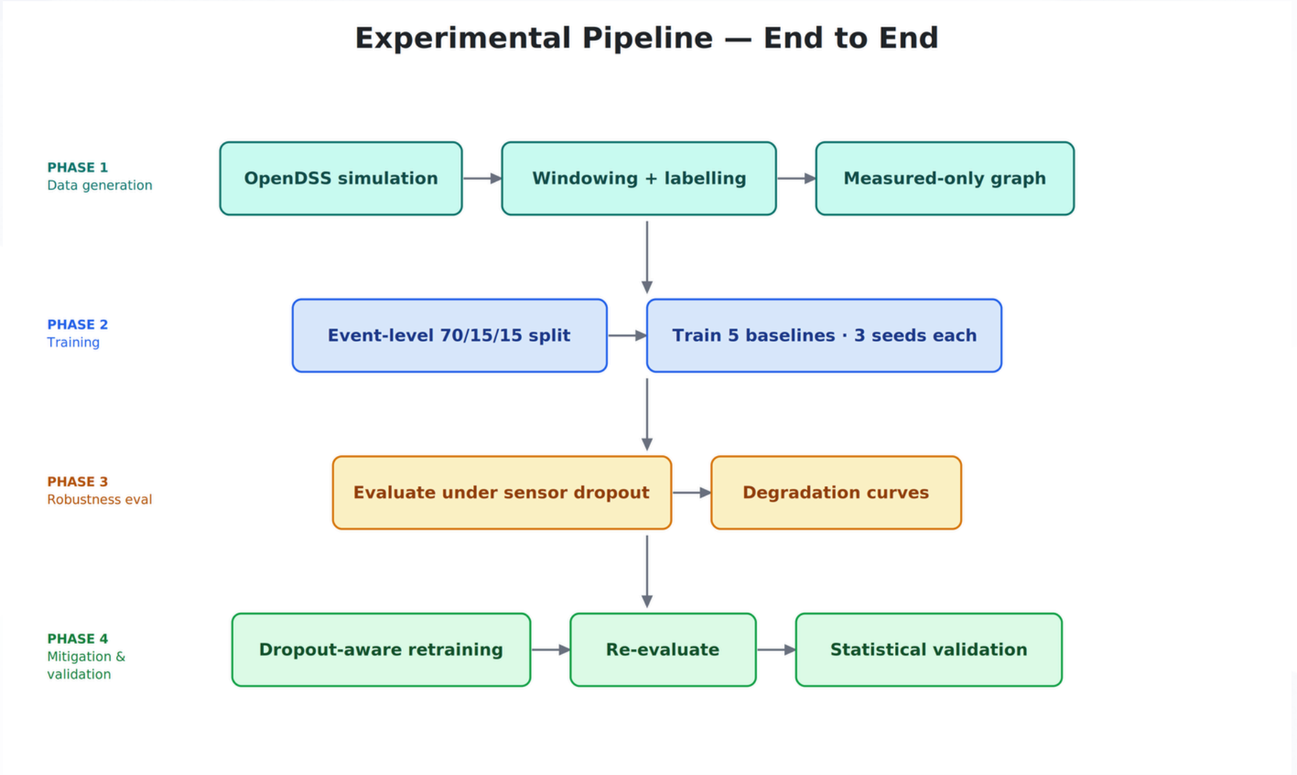
Model Architecture

- The GNN block is the only thing that differs across models — so any accuracy gap is due to spatial aggregation, nothing else
- Five choices for that block: GRU (no graph, baseline) · RGCN · RGSAGE-Mean · RGSAGE-Max · RGATv2
- Trained identically: 30 epochs · seeds 42 / 43 / 44 · single T1000 8 GB
- Declared deviations from paper: depth 4 (paper used 1) · 1.65 M windows (paper reports 2.5 M)

Every model shares the same backbone — only the graph layer changes



Workflow



- One frozen dataset and one frozen split are reused by every model
- Dropout masks are paired — every model sees the identical dropped nodes

Eval Stategy

- Metric: soft-voted macro-F1 (26 classes), plus per-resistance breakdown
- Robustness: drop $k \in \{0,1,2,5,10\}$ μPMUs at inference, k=10 is 40 % of the fleet
- Summary measures: AUDC (area under retention curve) and retention at k=10
- Every point = 3 training seeds × 3 mask seeds
- Paired t-tests with Holm correction + cluster bootstrap over 6,175 events

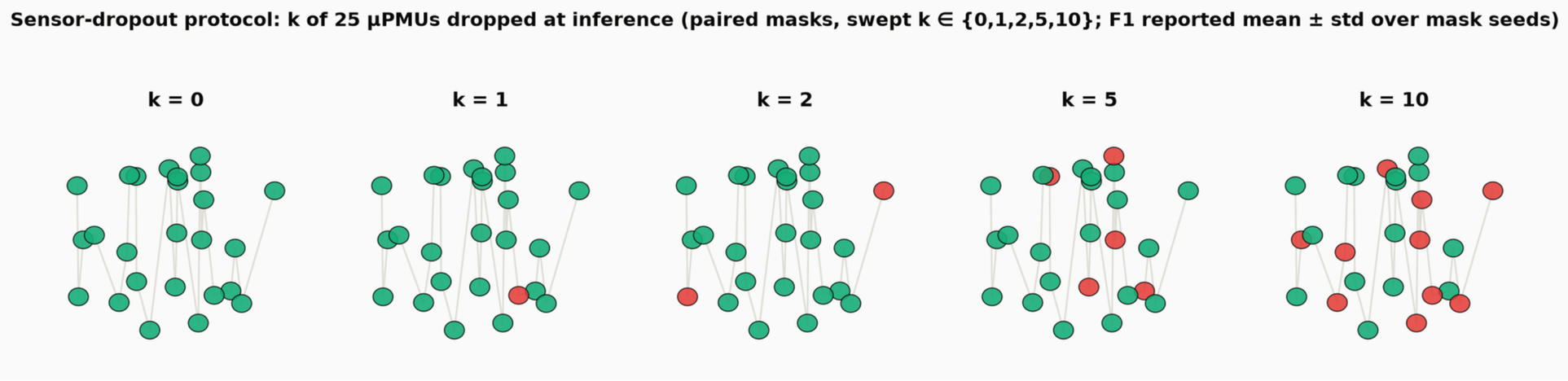
Results - All sensors

- Soft-voted macro-F1 on the test set, **mean ± std over seeds 42/43/44**:

Model	Event macro-F1	Per-window F1	F1 @ 0.1 Ω	F1 @ 1 Ω	F1 @ 10 Ω
GRU (no graph)	0.286 ± 0.009	0.150	0.410	0.292	0.105
RGATv2	0.522 ± 0.087	0.460	0.630	0.574	0.312
RGCN	0.712 ± 0.017	0.601	0.874	0.792	0.428
RGSAGE-Mean	0.786 ± 0.023	0.660	0.906	0.843	0.587
RGSAGE-Max	**0.846 ± 0.007**	**0.718**	**0.949**	**0.893**	**0.676**

- **The graph matters: +0.56 F1 from GRU → RGSAGE-Max. Topology is doing real work.**
- **Max aggregation wins — a fault is a localized extremum, and mean-pooling washes it out.**
- **Accuracy falls with fault resistance (0.949 → 0.676 at 10 Ω) — high-Z faults are the hard case.**
- **No-fault precision ≥ 0.97 and recall ≈ 1.00 for every model → no false alarms.**
- **Best-validation epoch was 27.7–28.7 of 30 for RGCN/RGSAGE-***

Robustness and Evals



- At **inference only**, drop a uniformly random subset of *k* µPMU nodes: zero all 6 channels of each dropped node (policy = zero_messages).Training is untouched.
- k ∈ {0, 1, 2, 5, 10} out of 25 nodes (i.e. up to 40 % of the fleet offline).
- 3 mask seeds × 3 training seeds for every point; error bars = std across training seeds.
- Two summary metrics:
- AUDC — normalized area under the degradation curve (1.0 = no degradation at all).
- Ret.@10 — F1 at k=10 as a fraction of F1 at k=0 (retention).

Removing Sensors Destroy every model

Model	k=0	k=2	k=5	k=10	Retained
RGSAGE-Max	0.846	0.590	0.236	**0.048**	**5.7 %**
RGSAGE-Mean	0.786	0.647	0.393	0.108	13.8 %
RGCN	0.712	0.610	0.418	**0.164**	**23.1 %**
RGATv2	0.522	0.444	0.302	0.157	30.1 %
GRU	0.286	0.252	0.111	0.051	17.9 %

- Zero out k μPMUs at inference only — training is untouched
- k = 10 of 25 means 40 % of the fleet is offline
- The most accurate model is the most fragile — RGSAGE-Max keeps just 5.7 %, RGATv2 keeps 30.1 %

Dropout Aware Training

- The fix isn't a new model — it's teaching the old one what failure looks like.

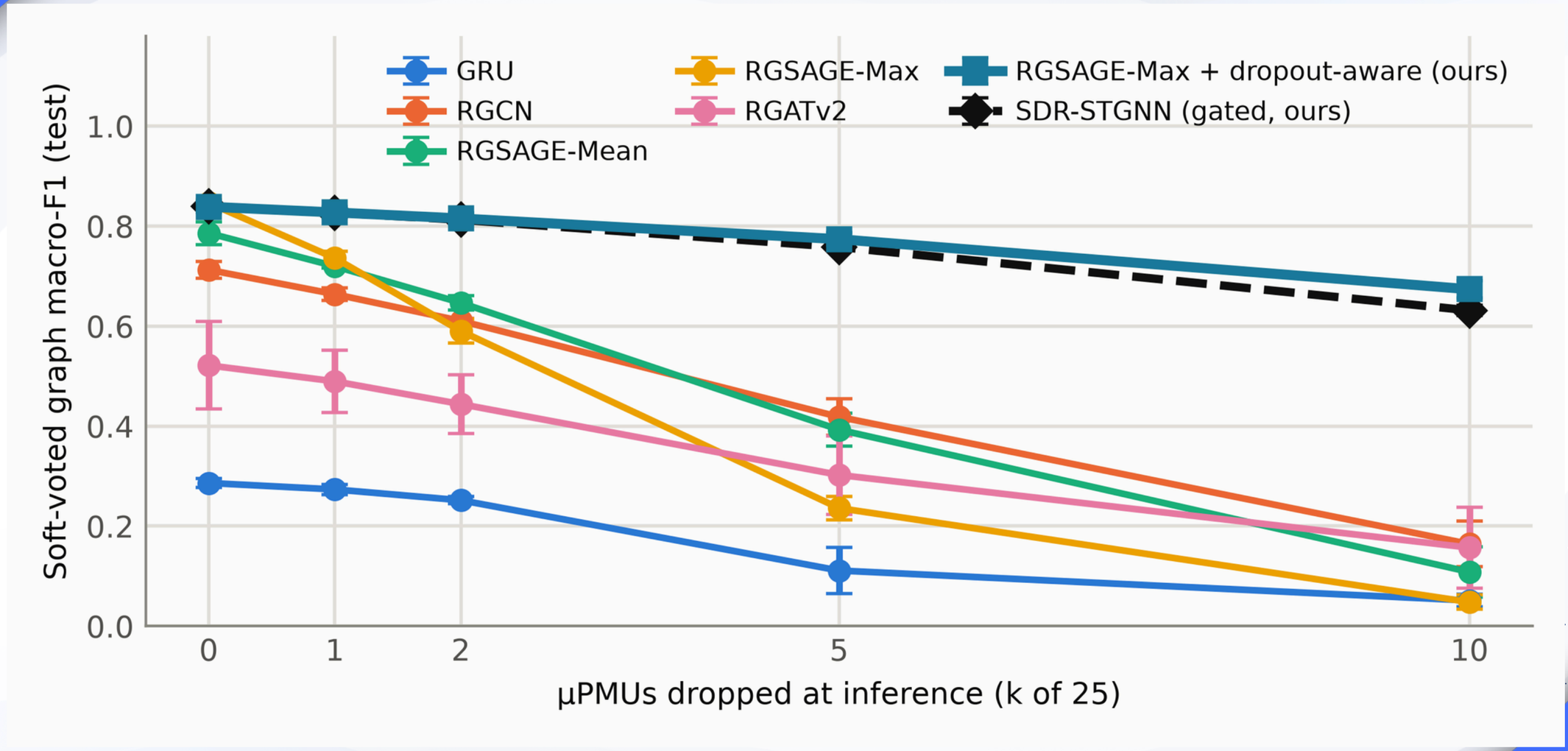
for each training batch:

k ~ U{0 .. k_max} S ~ random subset of k nodes

zero all 6 channels of nodes in S

train as usual

- It's a data augmentation — not an architecture change; every model keeps its exact structure
- Why zeroing matters: mask channels are static, and only 14 of 25 nodes are 3-phase
- Dropping a node forces its mask to (0,0,0) — a pattern the model never sees in normal training
- Dropout-aware training puts that failure pattern into the training distribution
- Cost: identical parameters, identical latency, +2.7 % training time


GRU
RGCN
RGSAGE-Mean
RGSAGE-Max
RGATv2
RGSAGE-Max + dropout-aware (ours)
SDR-STGNN (gated, ours)
Soft-voted graph macro-F1 (test)
1.0
0.8
0.6
0.4
0.2
0.0
0
1
2
5
10
μPMUs dropped at inference (k of 25)

Dropout Aware Training Results

Mitigation (RGSAGE-Max)	k=0	k=10	AUDC	Ret@10
Standard training	0.846	0.048	0.402	5.7 %
Mean-imputation	0.846	0.099	0.476	11.8 %
Dropout-aware (ours)	0.839	**0.674**	**0.913**	**80.3 %**
SDR-STGNN (gated)	0.839	0.631	0.892	75.2 %

- +0.626 macro-F1 at 40% loss over standard training (p_Holm = 0.003).
- Clean accuracy essentially unchanged (0.839 vs 0.846 — not significant).
- Free: identical parameters (95,258), identical latency (0.640 ms), only +2.7% training time.
- Imputation barely helps — reconstructed values still go to a model never trained to distrust them.

Dropout Aware Training over all Architectures

Model	Standard	Dropout-aware
RGSAGE-Max	5.7 %	80.3 %
RGSAGE-Mean	13.8 %	81.9 %
RGCN	23.2 %	87.0 %
RGATv2	31.0 %	91.4 %
GRU	17.9 %	25.4 %

- All four spatial GNNs jump to 80–91% retention (AUDC 0.40–0.63 → 0.91–0.96).
- GRU gains least — with no message passing, there's nowhere to route around a dark node.
- Clean-accuracy cost isn't uniform: ~0 on RGSAGE-Max, but ~0.1 F1 on RGCN and RGSAGE-Mean.

SDR-STGNN An Honest Negative Result

- We also built an availability-gated architecture that structurally silences dropped sensors.
- It matches but does not beat the training fix: AUDC 0.892 vs 0.913, Ret@10 75.2% vs 80.3% ($\Delta = 0.021$, $p = 0.098$).
- And it costs 18% more inference latency (0.755 vs 0.639 ms) for zero robustness gain.
- Why: silencing every dropped node also cuts the multi-hop paths that route information around a failure.
- Takeaway: dropout robustness is best bought in training, not baked into the architecture.

The fix survives Harder Failure Modes.

- **Targeted (worst-case) — drop the µPMUs nearest the fault:**
- Hurts everyone more: standard RGSAGE-Max falls to 0.448 at k=1 vs 0.736 random (3.6× larger drop).
- Dropout-aware is penalized too (AUDC 0.913 → 0.747) but still retains far more (+0.31 to +0.47 F1, $p \leq 0.008$).
- From k=2 up, its worst-case score beats the standard model's average-case score.

- **Persistent (per-event) — same sensors dark all event:**
- No effect — no curve moves by > 0.003 macro-F1 (AUDC 0.913 → 0.913).
- Because the decision is per-window, not per-event.

Conclusion

- A measurement: clean accuracy doesn't predict robustness — the strongest model degraded the fastest (5.7% retained vs RGCN's 23.2%).
- A fix: dropout-aware training restores retention 5.7% → 80.3% at zero parameter/latency cost, across all five architectures.
- A negative result: an availability-gated architecture only matches it.
- Robust to imputation, worst-case, and persistent loss.
- Practical: an operator can lose ~10 of 25 μPMUs and keep macro-F1 > 0.67 — no new hardware.
- Future work: correlated/structured failures, larger and reconfigured feeders, adaptive gating.
- This paper was submitted to ICMLDE (Dropout-Aware Training for Sensor-Robust Fault Localization with STGNNs — ICMLDE / Procedia Computer Science)

References

[1] Karabulut, Manna & Develder (2026). Robustness of spatio-temporal GNNs for fault location in partially observable distribution grids. arXiv:2604.20403.

[2] Krishnathevar & Ngu (2012). Generalized impedance-based fault location for distribution systems. IEEE Trans. Power Delivery 27(1):449–451.

[3] Liu et al. (2023). A novel fault location method based on traveling wave for multi-branch distribution network. Electric Power Systems Research 224:109753.

[4] Livani, Evrenosoglu & Centeno (2013). A machine learning-based faulty line identification for smart distribution network. NAPS:1–6.

THANK YOU